Urban Microclimate Modelling with Machine Learning, Deep Learning and a Hybrid Surrogate: From Layout Summaries to Wind Maps

Suhaani Garg, Aarya Srivastava, Palak, Jashvitha Lakshmi Omkaram, Samarth Sangtani
Newton School of Technology, Rishihood University
Project 6 – CSAI Project Report (Phase 1: Machine Learning, Phase 2: Deep Learning)

***Abstract*—Wind between buildings affects comfort, heat and air quality, but simulating it with Computational Fluid Dynamics (CFD) is slow. We build fast surrogate models on the UrbanTALES dataset of 224 idealized building-array simulations. In Phase 1 (Machine Learning) we describe each layout with a few summary numbers and predict one value, the canopy-mean wind speed; XGBoost reaches a mean absolute error (MAE) of 0.232 m/s ($R^2 = 0.90$) when whole layout families are held out. In Phase 2 (Deep Learning) we feed the layout as an image to a compact U-Net that predicts the full pedestrian-level wind map, and we test a hybrid that adds the Phase 1 prediction as an extra input channel. The U-Net reproduces the main wake patterns on unseen layouts, but a simple CNN does as well on our test set, and the hybrid gives no measurable gain. We report all results, including the negative ones, with their limitations. This study is about surrogate accuracy and uncertainty, not safety approval of real buildings.**

***Index Terms*—urban microclimate, surrogate model, U-Net, XGBoost, CFD, grouped cross-validation, hybrid learning**

I. Introduction

Wind between buildings decides how comfortable a street feels, how quickly heat builds up and how long polluted air stays trapped. Engineers calculate this wind with Computational Fluid Dynamics (CFD), which simulates the air around every building. CFD is accurate, but one layout can take hours or days on a powerful computer, so designers cannot test hundreds of ideas.

A *surrogate model* is a fast stand-in for such a simulation: it is trained on finished CFD results and then answers new questions almost instantly. This project builds and compares surrogates for simplified city layouts (regular grids of blocks) using the UrbanTALES dataset [1]. We worked in two phases:

- **Phase 1 (ML):** describe each layout with a few numbers (building density, height statistics, wind direction) and predict *one number*, the average wind speed in the street canopy.
- **Phase 2 (DL):** give the model the layout as a *picture*, predict the *whole wind map* at pedestrian height, and test a hybrid that feeds the Phase 1 prediction into the network.

The dataset has no temperature output, so we model wind only. Table I lists the key terms in plain words.

TABLE I
KEY TERMS IN PLAIN WORDS

CFD / LES	Computer simulation of air flow; large-eddy simulation is a detailed, slow type that produced our data.
Canopy	Air layer from the ground up to the average building height.
Family	Layouts sharing an arrangement type and building density; a whole family always stays on one side of a split.
Held-out	Data never seen while learning, used only for a fair test.
Field / map	A wind value at every grid cell, like a picture, not a single number.
U-Net	Image network that squeezes the picture, expands it back, and uses shortcuts to keep fine detail.
MAE / RMSE	Average prediction error, and a version that punishes big misses more. Lower is better.

A. Why we chose this project

- **Real-world value:** quick wind estimates help compare street layouts early in planning, long before an expensive CFD run.
- **A natural ML-to-DL story:** the project brief asks for a layout-summary baseline first and a spatial model second, so we could see what each approach can and cannot do, and apply what we learned in the course (feature engineering, careful validation, CNNs, U-Nets).
- **Results you can see:** a wind map and an error map show immediately whether a model has understood the flow.
- **A solid dataset:** UrbanTALES supplies 224 ready simulations with table values and full velocity maps, so our time went into modelling instead of running OpenFOAM.
- **Honest evaluation:** layouts in a family look alike, so a careless split hides mistakes; designing a fair test was part of the challenge.

II. Related Work and Key Papers

Our design follows a few papers closely.

- **Thuerey et al. [3]** trained a U-Net to predict velocity and pressure fields around airfoils from the shape plus flow conditions given as constant input channels, tested it on shapes it had not seen, and added no built-in physics. We use the same recipe: a U-Net with geometry and constant condition channels (wind direction, friction velocity), evaluated on unseen geometries.

- **Roberts et al. [4]** show that ordinary random cross-validation overstates accuracy when data come in related groups, and recommend holding out whole groups; they also warn that this can force a model to extrapolate. This is why we hold out whole layout families, and it explains why our test families at the density extremes are hard.
- **Chen and Guestrin [5]** introduced XGBoost, which builds trees one after another with shrinkage, subsampling and L_2 regularization. Our Phase 1 XGBoost uses these controls (learning rate 0.03, depth 2, subsample 0.85, column sample 0.9).
- **Ronneberger et al. [2]** proposed the U-Net, an encoder–decoder with skip connections, which is our Phase 2 backbone.
- **Brunton et al. [6]** and **Farimani et al. [7]** review and demonstrate learning of fluid and transport solutions, and stress careful generalization tests. The faculty brief's title for Farimani et al. differs from the verified 2017 paper, so we cite the verified one.
- **Nazarian et al. [1]** provide the UrbanTALES large-eddy simulation data we learn from.

III. The Data: Small for ML, Much Bigger for DL

Both phases use the same 224 simulations: 112 layouts, each run with two wind directions (0° and 45°). The layouts form **16 families**: an arrangement type and one of 8 density levels from 0.0625 to 0.64, giving 14 cases per family. What changed is how much information we took from each simulation.

For Phase 1, one number per case was enough, so we used a small table: the building-height metadata and one flow-data table (224 rows). The sample ZIP archives supplied at that stage held only 32 members each, so they were used for checks only. For Phase 2 we needed the full maps. The complete UrbanTALES package provides **224 files for each of four file types**: pedestrian-level NetCDF velocity files, topography (building) files, time-series NetCDF files and vertical-profile CSV files. We verified that all four types cover all 224 cases, and used the **pedestrian NetCDF** files (the velocity maps we predict) and the **topography** files (the building maps we input). The time-series and profile files were checked but not used.

The number of simulations stayed at 224, but each grew from a single target value to about 68,000 air cells with two velocity values. That is why Phase 1 could use a small table while Phase 2 needed the complete field and topography files and an image model. One caution: the NetCDF velocity variables carry no unit attribute, so we report field errors in stored units and do not call them m/s.

IV. Phase 1: The Machine Learning Pipeline

The ML work ran through eight notebooks:

- **ML_00, ML_01:** problem framing, literature, a clean dataset and a data dictionary; we checked the 224/112/16 structure and audited the sample ZIPs.
- **ML_02 (EDA):** wind speed is right-skewed (Fig. 1); density is the strongest driver (denser blocks, slower wind); 0° wind is faster on average (2.24 m/s) than 45° (1.86 m/s).
- **ML_03:** leakage audit; IDs and answer-leaking columns removed; squared density, height spread, wind-direction sine and cosine added.
- **ML_04:** every family is held out whole in 5-fold GroupKFold, following [4].
- **ML_05–ML_07:** Ridge Regression, Random Forest and XGBoost [5], compared with a mean-only baseline.

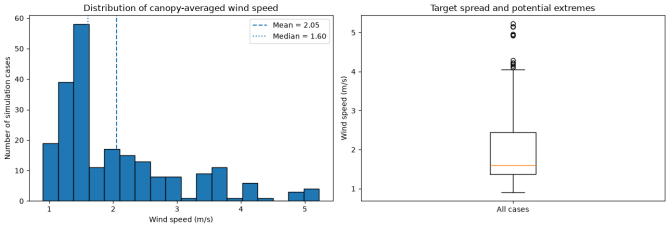


Fig. 1. Distribution of the canopy-mean wind speed target (ML_02): most cases are slow, with a long tail of fast ones.

What we learned. XGBoost was best (MAE 0.23 m/s, $R^2 = 0.90$), but its gap to Random Forest is not reliable: the paired interval (-0.016 to 0.069 m/s) includes zero. Density features dominate: plan-area density and its square carry about 73% of XGBoost's importance, followed by horizontal arrangement (8%) and wind direction (12%). The split matters: when only whole layouts were held out, with other layouts of the same family left in training, Random Forest error fell to 0.065 m/s, which is over-optimistic. The main limit of Phase 1 is that a handful of summary numbers cannot describe wakes or street-canyon effects.

V. How Deep Learning Helped

Phase 1 can say "the average wind speed in this canopy is about 1.6 m/s". Phase 2 shows *where* the air is calm or fast around every building. The deep model sees the layout as an image, so local patterns such as the sheltered wake behind a block can be learned instead of averaged away, and error maps on unseen layouts show exactly where the surrogate fails. The two phases answer different questions, so their errors cannot be compared directly: Phase 1 error is in m/s for one value, Phase 2 error is in stored units over a whole map.

VI. Phase 2: The Deep Learning Pipeline

A. DL_01: Data preparation

For each of the 224 cases we read the pedestrian velocity components (`uped`, `vped`) and the building height map, rotated the arrays to image orientation, and built a **valid-air mask**: a cell counts only if it is not a building and both velocities are finite. No resampling was applied, so the 0.5 m grid is kept. All 224 cases were matched against the file lists and saved with a manifest. Normalization statistics (mean and standard deviation of u and v, a building-height scale of 37 m, friction velocity) were computed on **training cases only** (7.68 million air cells), so nothing leaks from validation or test.

TABLE II
WHAT ONE CASE LOOKS LIKE IN EACH PHASE

	Phase 1 (ML)	**Phase 2 (DL)**
What one case is	One row of a table	One 2D wind map plus a building map
Files used	Metadata (building heights) and the flow-data table; 32-member sample ZIPs for checks only	224 pedestrian NetCDF + 224 topography files (224 time-series NetCDF and 224 profile CSV files also present, not used)
Input	14 layout features (11 numeric, 3 categorical)	5 image channels: building height, wind direction (sin, cos), friction velocity, domain mask
Target	One value: canopy-mean wind speed (0.90 to 5.22 m/s, mean 2.05)	Two values at every air cell: u and v velocity at 1.75 m
Size	224 rows	Grids from 120×200 to 384×768 cells (0.5 m spacing); 5,184 to 257,472 air cells per case; 15.2 million in total
Why this much	One number per case needs only a summary table	A map needs the position of every building and the wind at every cell

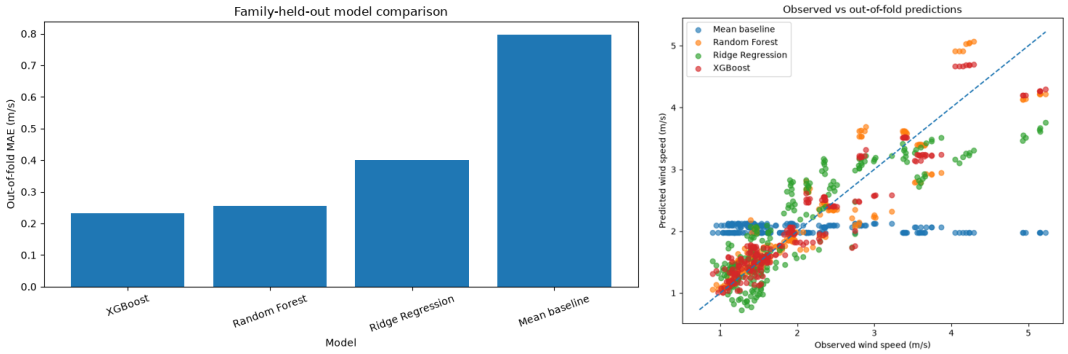


Fig. 2. Left: out-of-fold error by model (ML_07). Right: observed versus predicted wind speed for all four models.

TABLE III
FAMILY-HELD-OUT RESULTS (224 CASES, 16 FAMILIES)

Model	MAE	RMSE	R^2	MAE 95% interval
Mean baseline	0.798	1.011	−0.03	0.566–1.082
Ridge Regression	0.400	0.507	0.74	0.312–0.500
Random Forest	0.256	0.380	0.85	0.176–0.337
XGBoost	**0.232**	**0.309**	**0.90**	0.182–0.288

MAE and RMSE in m/s.

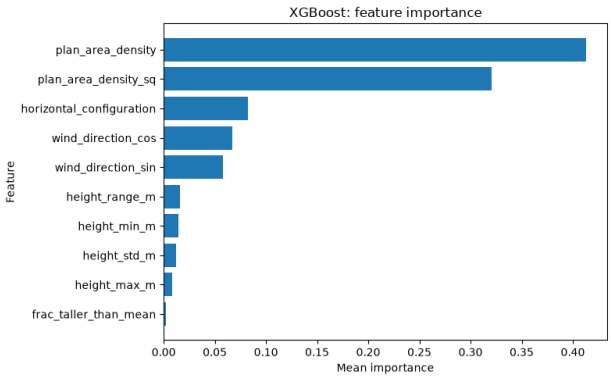


Fig. 3. XGBoost feature importance (ML_07): the two density features dominate, then horizontal arrangement and wind direction.

B. DL_02: Literature and architecture

We chose a compact **U-Net** [2]: an encoder squeezes the image to see the big picture, a decoder expands it back, and skip connections carry fine detail. Following [3], it takes **5 input channels** (building height, sine and cosine of wind direction, friction velocity, domain mask) and gives **2 output channels** (u, v). It uses GroupNorm, GELU, a dropout layer in the middle and 486,706 parameters. A much smaller **compact CNN** (32,570 parameters, no skips) is the baseline. The loss is a **masked mean squared error**, so building cells never count.

C. DL_03: Training and regularization

Whole families were split into **10 for training (140 cases), 3 for validation (42 cases; S1111, S25, S64) and 3 for testing (42 cases; A0625, A1111, A4444)**. Training used batch size 1, AdamW (learning rate 10^{-3}, weight decay 10^{-4}), gradient clipping at 1.0, seed 42 and Apple MPS hardware. Against overfitting we used dropout, weight decay, clipping and early stopping (patience 7 for the U-Net, 5 for the CNN). The U-Net ran all 25 epochs (best validation loss 0.241); the CNN stopped at epoch 13 (best 0.517). Fig. 6 shows the usual small-data picture: training loss keeps falling to about 0.09 while validation loss flattens near 0.25.

D. DL_04: Validation, error maps and ablations

All models were scored once on the three untouched test families, over **5.16 million valid air cells**. We report field MAE, RMSE and normalized RMSE (RMSE divided by the range of the target). Ablations remove inputs from the same U-Net.

Fig. 7 shows three held-out cases. The U-Net reproduces the main pattern, with slow, sheltered air behind each block and

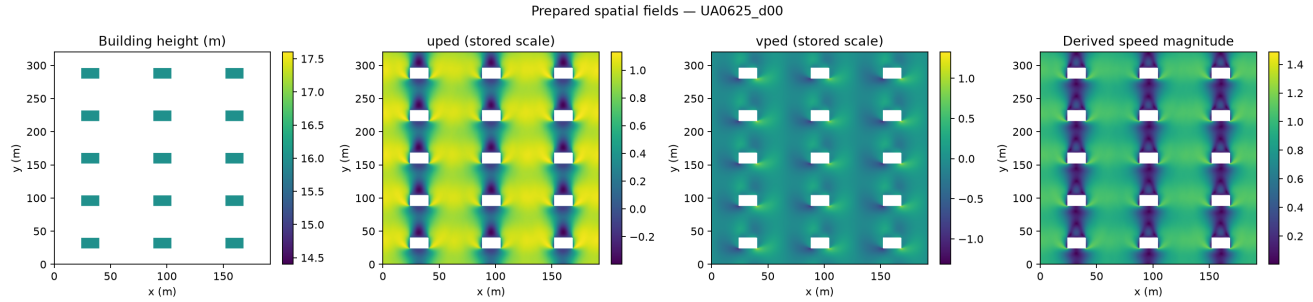


Fig. 4. Prepared data for case `UA0625_d00` (DL_01): building height, stored u and v velocity, and the derived speed.

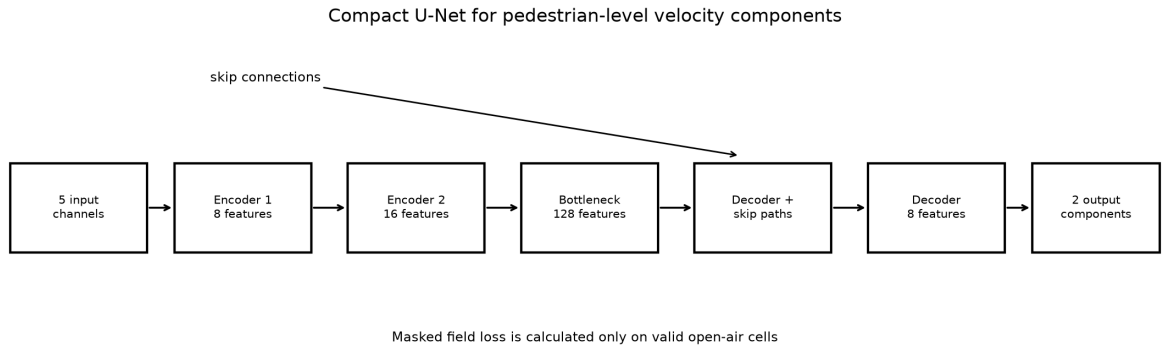


Fig. 5. Compact U-Net for the pedestrian-level velocity components (DL_02). The loss is computed on valid air cells only.

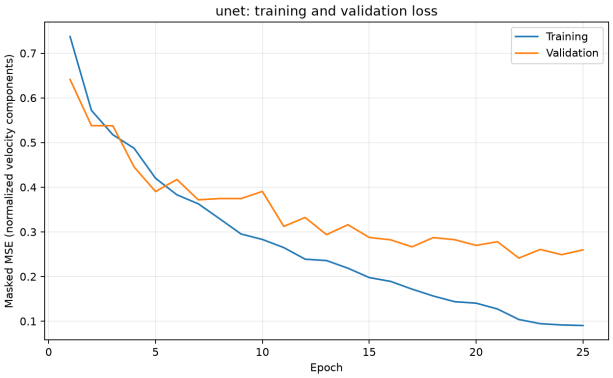


Fig. 6. U-Net learning curve (DL_03): masked MSE over 25 epochs.

TABLE IV
TEST RESULTS ON HELD-OUT FAMILIES (STORED UNITS)

Model	MAE	RMSE	NRMSE	Epochs
U-Net (5 channels)	0.2438	0.3416	0.107	25
Compact CNN	0.2238	0.2839	0.089	13 (early stop)
Ablation: geometry only	0.2815	0.3716	0.117	10
Ablation: no friction vel.	0.2623	0.3576	0.112	10

faster flow in the gaps beside it. Its picture is streakier than the truth, and errors sit in the streets right around buildings, where the wind changes quickly. Error also depends on the family: U-Net MAE was **0.300 for the sparsest layouts (A0625), 0.161 for A1111 and 0.081 for the densest (A4444)**. Our test families are the density extremes, so this is a demanding test.

Two honest points. First, the simple CNN scored better than the U-Net on the test families (MAE 0.224 vs 0.244) although the U-Net was better on validation, so with three test families we cannot call the U-Net the winner. Second, the ablations used 10 epochs against 25 for the full model, so their gaps show direction (more inputs helped) but not an exact size.

E. DL_05: The hybrid ML + DL model

Following the project brief, we give the U-Net the canopy-mean speed predicted by the Phase 1 XGBoost as an extra constant 6th channel, hoping this global hint helps a network that only sees local windows. To stay leak-free, each **training** case's value comes from models that never saw its family (out-of-fold), and validation and test values come from the average of those fold models. Our first attempt crashed the notebook and had problems (wrong scaling of the channel, no fixed seed, no control), so we rebuilt it: XGBoost runs in its own process, the channel is scaled with the true training target, every run has a fixed seed, and we added a **shuffled-channel control** that feeds the same values to the wrong cases. If the real channel is useful it must beat that control.

Result. The hybrid did **not** improve the surrogate. It beat the base U-Net on only 16 of 42 test cases and was worse on the sparsest family, where XGBoost itself must extrapolate (test $R^2 = 0.74$, against 0.90 in Phase 1 cross-validation). The shuffled control did as well as the real channel, so differences of a few percent are within ordinary run-to-run variation

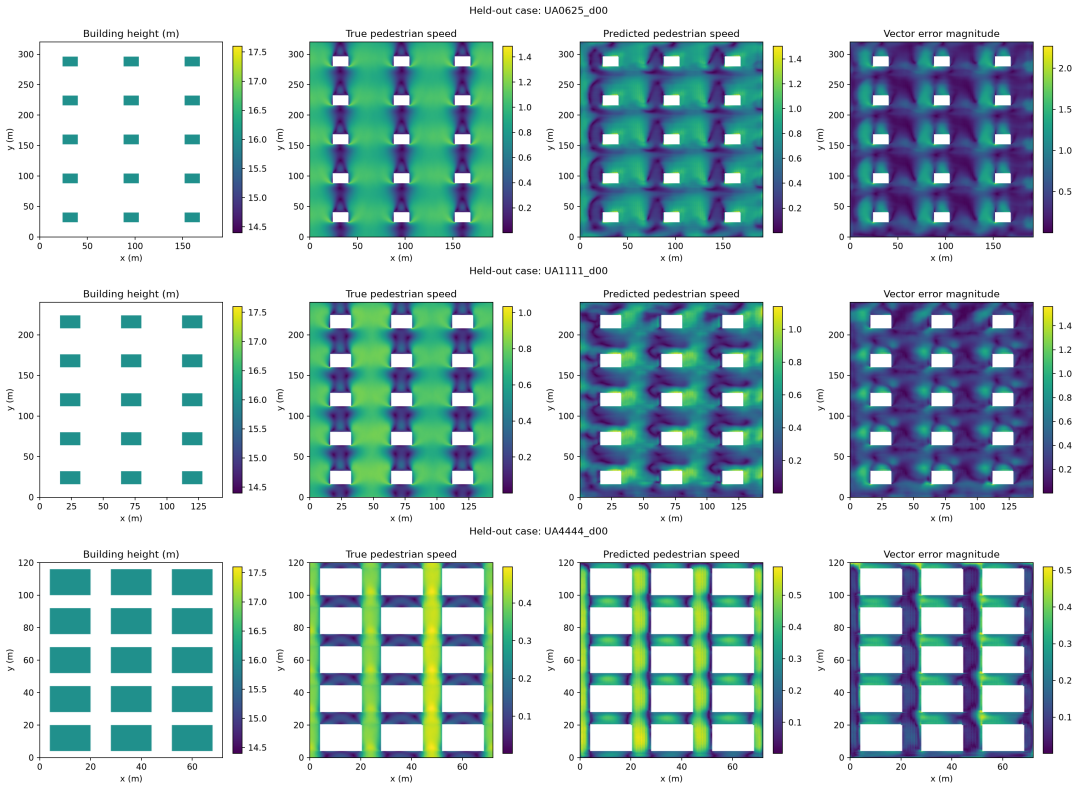


Fig. 7. Held-out test cases UA0625, UA1111 and UA4444 (DL_04). Left to right: building height, true speed, predicted speed, vector error magnitude.

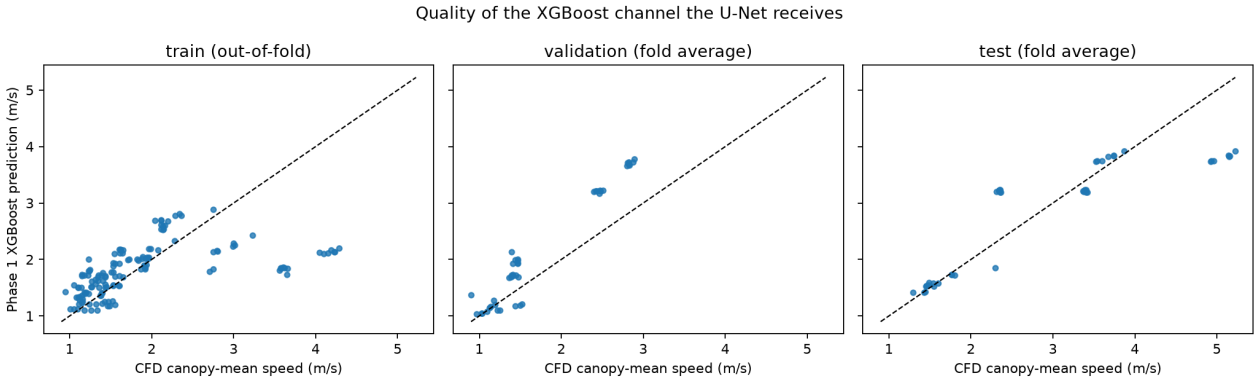


Fig. 8. Quality of the XGBoost channel the U-Net receives (DL_05): out-of-fold on train, fold average on validation and test.

TABLE V
HYBRID EXPERIMENT (SEED 42, TEST FAMILIES)

Model	MAE	RMSE	Change vs base
Base U-Net	0.2438	0.3416	reference
Hybrid (real XGBoost channel)	0.2527	0.3517	3.6% worse
Shuffled-channel control	0.2386	0.3426	2.1% better

(more seeds are needed to confirm). We report the hybrid as a **correctly implemented, controlled experiment with a negative result**.

VII. FINDINGS AND LIMITATIONS

Layout summaries predict the average canopy wind speed fairly well (XGBoost $R^2 = 0.90$ on unseen families), mainly through building density. A U-Net turns a building map into a full wind map for unseen layouts and its error maps show where it fails, but a tiny CNN did as well on our test set. Adding the Phase 1 prediction as an extra channel gave no measurable benefit.

Limitations. The data are idealized block arrays with two wind directions and 16 families. The test set is three families at the density extremes. All DL results use one seed, and the ablations use a shorter training budget. The model has

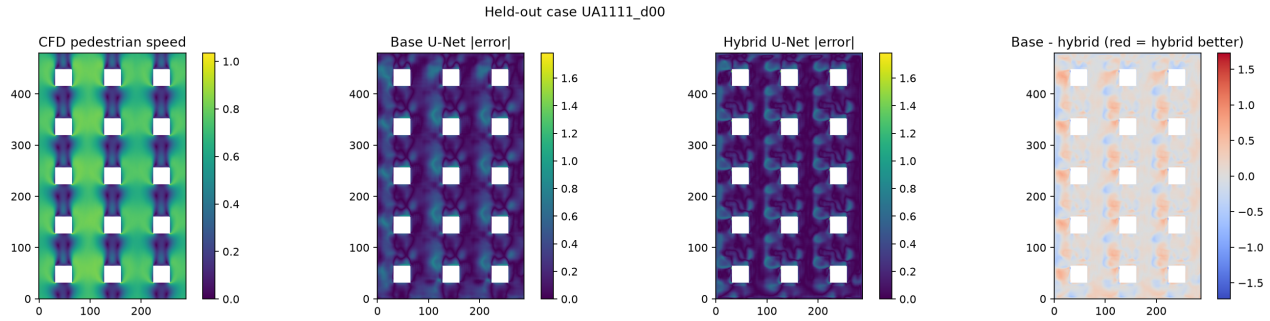


Fig. 9. Held-out case `UA1111_d00` (DL_05): true speed, base U-Net error, hybrid U-Net error, and base minus hybrid (red means the hybrid is better). The last panel is almost flat, so the two models behave alike.

no physics built in and predicts no temperature. **Next steps:** repeat with seeds 42, 43 and 44, retrain the ablations with the same budget as the full model, cross-validate the deep models over families, and try a stronger conditioning method than a constant channel.

Reproducibility. The repository holds the ML notebooks (ML_00 to ML_07), the DL notebooks (DL_01 to DL_05), a shared `dl_src` module (data loading, models, training, metrics, hybrid channel), and the saved tables, figures and model files in `results` and `dl_results`. All splits are stored in manifest files, normalization statistics are fitted on training cases only, and runs use a fixed seed.

VIII. Team Contributions

Everyone contributed to the literature review and this report, and each member is prepared to explain the whole pipeline.

References

[1] N. Nazarian et al., "UrbanTALES: A Large-Eddy Simulation Dataset for Urban Canopy Layer Turbulence and Parameterization," *Bull. Amer. Meteorol. Soc.*, vol. 106, no. 12, pp. E2461–E2478, 2025, doi: 10.1175/BAMS-D-25-0061.1.

[2] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.

[3] N. Thuerey, K. Weißenow, L. Prantl and X. Hu, "Deep learning methods for Reynolds-averaged Navier–Stokes simulations of airfoil flows," *AIAA J.*, vol. 58, no. 1, pp. 25–36, 2020, arXiv:1810.08217.

[4] D. R. Roberts et al., "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017, doi: 10.1111/ecog.02881.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.

[6] S. L. Brunton, B. R. Noack and P. Koumoutsakos, "Machine learning for fluid mechanics," *Annu. Rev. Fluid Mech.*, vol. 52, pp. 477–508, 2020, doi: 10.1146/annurev-fluid-010719-060214.

[7] A. B. Farimani, J. Gomes and V. S. Pande, "Deep learning the physics of transport phenomena," arXiv:1709.02432, 2017.

TABLE VI
PHASE 1 (ML) AND PHASE 2 (DL) RESPONSIBILITIES

Notebook	**Team member**	**Responsibility**
ML_01	Suhaani Garg	Dataset understanding and data dictionary
ML_02	Aarya Srivastava	EDA and target definition
ML_03	Jashvitha Lakshmi Omkaram	Leakage audit and feature engineering
ML_04	Jashvitha Lakshmi Omkaram	Grouped split and preprocessing
ML_05	Palak	Ridge regression
ML_06	Samarth Sangtani	Random Forest and XGBoost
ML_07	Suhaani Garg	Evaluation, comparison and diagnostics
DL_01	Suhaani Garg	Dataset audit and data preparation
DL_02	Aarya Srivastava	Literature review and architecture
DL_03	Palak	Training and regularization
DL_04	Jashvitha Lakshmi Omkaram	Technical validation, error maps, ablations
DL_05	Samarth Sangtani	Hybrid ML + DL integration